IGA-SRAM: A Compact SRAM-Based IMC Engine for Binary/Ternary Deep Neural Networks With ADC-Less and Error-Aware Training Algorithm

Shin-Uk Kang, Dong-Hyun Lee, *Graduate Student Member, IEEE*, Min-Gwon Song, Mingoo Seok, *Senior Member, IEEE*, and Min-Seong Choo, *Senior Member, IEEE*

***Abstract*—This work presents a compact 6T SRAM-based analog in-memory computing (A-IMC) macro for energy-efficient inference of binary and ternary neural networks (BNNs/TNNs). While conventional A-IMC architectures suffer from significant area and power overheads due to high-resolution analog-to-digital converters (ADCs), we propose an ADC-less computing scheme based on 1-bit sense amplifiers (SAs), supported by a novel training algorithm that mitigates quantization-induced performance degradation. To maintain hardware simplicity, we adopt a compact 6T bitcell structure and introduce dual word-lines (WLs) for efficient ternary input encoding. We further propose impulse Gaussian approximation (IGA), a differentiable quantization surrogate that enables stable and accurate gradient backpropagation in the presence of 1-bit partial sum quantization. Additionally, we introduce an error-aware training (EAT) method that leverages measured partial-sum (PS) error statistics from silicon to inject gradually increasing noise in the training phase, effectively compensating for hardware-induced variations. Fabricated in a 65 nm CMOS GP process, the proposed 128×128 A-IMC macro achieves 47.06 TOPS/mm^2 and CIFAR10 accuracy of 86.89% under 1.2/0.7 V operating voltages with ResNet20.**

***Index Terms*—1-bit ADC, 6T SRAM, ADC-less, binary, BNN, error-aware training (EAT), IMC, impulse Gaussian approximation (IGA), in-memory computing, MAC, multiplication-and-accumulation, neural network, ternary, TNN.**

I. Introduction

SRAM-BASED analog in-memory computing (A-IMC) has emerged as a promising solution for edge applications such as TinyML [1], where high area and energy efficiencies and low-latency operation are required [2]. Recent A-IMC studies [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13] have focused on accelerating binarized neural networks (BNNs) [14], [15], [16] and ternary neural networks (TNNs) [17], [18], leveraging their high memory efficiency and low computational cost, which are essential for edge applications. However, several design challenges remain for commercialization in practical scenarios. In A-IMC, the partial sum (PS) produced by the IMC array is handled as an analog voltage, necessitating dedicated analog-to-digital converters (ADCs) for every column to accumulate parallel PS results. Typically, the resolution of the ADCs within an A-IMC macro affects the overall computation precision [3]. As illustrated in Fig. 1(a), for flash ADC-based designs, higher ADC resolution help maintain the network performance at the cost of significant area and

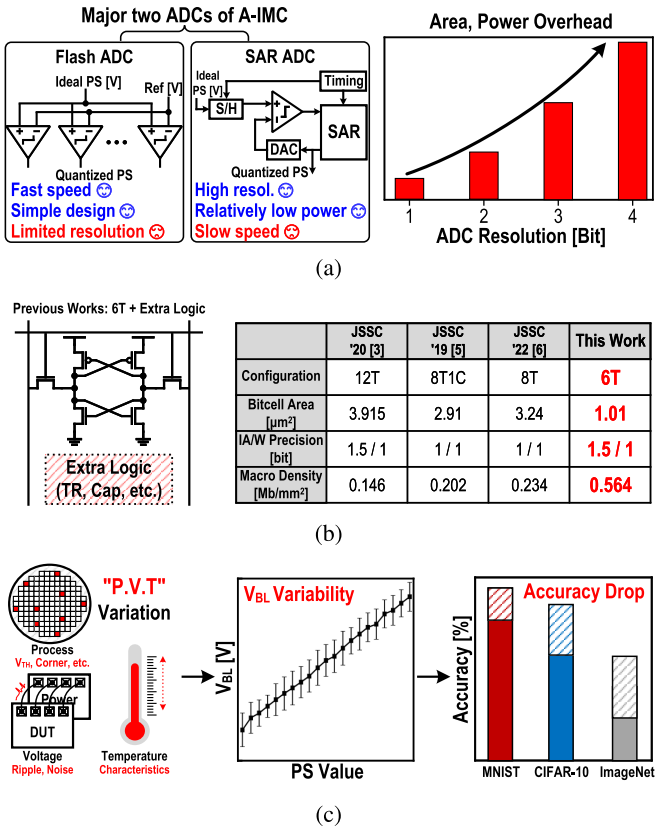


Fig. 1. Design challenges of analog A-IMC macro: (a) ADC's area/power overhead versus resolution, (b) large bitcell/macro size and (c) PVT variation.

Received 14 September 2025; revised 4 December 2025 and 9 February 2026; accepted 24 March 2026. Date of publication 27 March 2026; date of current version 15 June 2026. This work was supported by the Research Fund of Hanyang University under Grant HY-2025-2757. This article was recommended by Guest Editor L. M. B. Poehls. *(Shin-Uk Kang and Dong-Hyun Lee contributed equally to this work.) (Corresponding author: Min-Seong Choo.)*

Shin-Uk Kang is with the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon 34141, South Korea.

Dong-Hyun Lee and Min-Seong Choo are with the School of Electrical Engineering, Hanyang University, Ansan 15588, South Korea (e-mail: mschoo@hanyang.ac.kr).

Min-Gwon Song is with the Department of Electronic Engineering, Hanyang University, Seoul 04763, South Korea.

Mingoo Seok is with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA (e-mail: ms4415@columbia.edu).

Digital Object Identifier 10.1109/JETCAS.2026.3678317

2156-3357 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

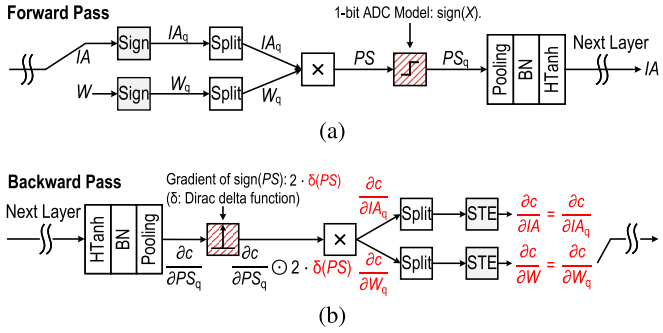


Fig. 2. Conventional training procedure of BNN with ADC quantization: (a) forward propagation and (b) backward propagation. The red text indicates gradient vanishing caused by the impulse gradient of the ADC function.

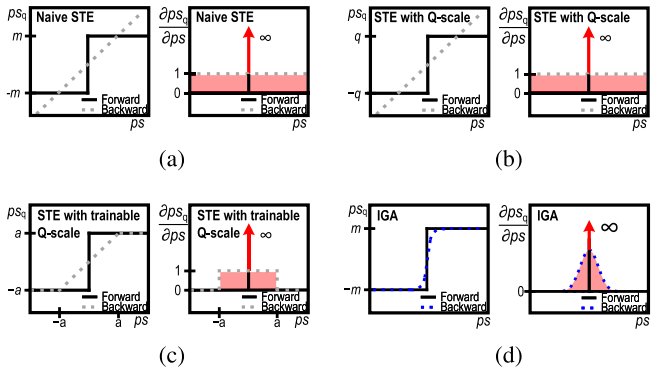


Fig. 3. Previous gradient approximation techniques for ADC-less training: (a) naive STE approach, (b) STE with quantization scaling [7], [8], (c) STE with trainable quantization parameter [9], [20], and (d) the proposed impulse Gaussian approximation (IGA) algorithm.

power overhead. For successive approximation register (SAR) ADC-based designs, a higher resolution further require more conversion cycles, leading to a reduction in throughput.

Prior A-IMCs [3], [5], [6] often incorporate extra transistors or capacitors into the bitcells to support binary/ternary multiplication and accumulation (MAC) operations as depicted in Fig. 1(b). However, this additional complexity leads to a significant increase in macro size, making them unsuitable for area- and energy-constrained edge AI applications. To address the trade-offs above, this work introduces an A-IMC that combines lightweight hardware with practical training algorithms to reduce additional circuitry [see Fig. 3(d)]. The proposed ADC-less training algorithm, impulse Gaussian approximation (IGA), uses a reliable surrogate gradient for the ADC's non-differentiable quantization function, thereby mitigating accuracy degradation and allowing the ADC resolution to be reduced to a single bit. And, to support both binary and ternary MAC operations in a compact macro area, we designed a 6T SRAM with separate wordlines (WLs) and a unified bitline (BL) [see Fig. 7] with an active area of 1.01 μm^2. Thanks to the simple bitcell design, we achieved 0.564 Mb/mm^2 of the macro density, exhibiting the highest area efficiency compared to previous state-of-the-art A-IMC works [3], [5], [6], as illustrated in Fig. 1(b).

In addition, we propose an error-aware training (EAT) framework. In general, the accuracy of DNN evaluated in A-IMC suffers from computing errors caused by hardware faults such as device mismatch and PVT variations [3], [4], [5], [6], [10], [11], [12], [19], as shown in Fig. 1(c). For example, the strength mismatch of the transistors in SRAM bitcells leads to variations in MAC voltages, which induce errors in the ADC outputs. Furthermore, voltage noise and subtle temperature variations also contribute to the computing errors. As these factors affect not only SRAM's operation, but also the other circuits in the macro (e.g., WL and BL drivers and ADCs), the precise on-chip calibration is challenging. The proposed error-aware training scheme successfully compensates for hardware faults and the resulting computation errors in the software domain, achieving 86.89% of the CIFAR10 accuracy with 1-bit sense amplifiers (SAs).

This work, termed IGA-SRAM, accelerates neural networks with binarized weights, including (i) BNNs with binarized activations and (ii) TNNs with ternarized activations. The three key contributions are:

- **Compact 6T SRAM:** supporting binary and ternary MAC operations without any additional logic or circuitry.
- **Hardware-software codesign:** ADC-less A-IMC architecture with the proposed IGA training algorithm.
- **Error-aware training (EAT):** warm-up error injection to improve classification accuracy.

The remainder of this manuscript is organized as follows. Section II describes the challenge of ADC-less training, as discussed in previous works, and introduces the proposed IGA algorithm. The proposed binary/ternary compatible 6T SRAM bitcell, operation principle, and the overall architecture of the 2 KB A-IMC engine are presented in Section III. Section IV provides measurement setup and measurement results, including the proposed error-aware training framework, followed by a conclusion in Section V.

II. IGA: ADC-Less Training Algorithm

A. Gradient Vanishing Problem and Previous Works

For ADC-less A-IMC architecture, the DNNs should be trained considering the analog-to-digital conversion process with a 1-bit ADC (i.e., ADC-less training) [7], [8], [9], [20], [21]. First, the ADC function is modeled with a sign function, which is a quantization function with 1-bit resolution. And the sign function is inserted after MAC operations in the forward pass of the training phase:

$$\textbf{Forward: } ps_q = m \cdot sign(ps), \tag{1}$$

as illustrated in Fig. 2(a). Here, ps denotes the partial sum, ps_q denotes the quantized partial sum, and m denotes the fan-in constraint (i.e., the row size of the array) of the A-IMC. In the backward pass, the derivative of the sign function $2 \cdot \delta(\cdot)$ is inserted:

$$\textbf{Backward: } \frac{\partial c}{\partial ps} = \frac{\partial c}{\partial ps_q}(2m \cdot \delta(ps)). \tag{2}$$

Here, c denotes the loss function, and $\delta(\cdot)$ denotes the Dirac delta function. However, the impulse gradient of the ADC causes a gradient vanishing problem, as indicated in Fig. 2(b). In previous studies [7], [8], [9], [20], various techniques

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

have been employed to alleviate this issue and enable stable training.

The most naive approach is to use a straight-through estimator (STE) [22] as illustrated in Fig. 3(a). In this case, the backward gradient is passed through as:

$$\textbf{Forward: } ps_{\mathrm{q}} = m \cdot sign(ps), \tag{3}$$

$$\textbf{Backward: } \frac{\partial c}{\partial ps} = \frac{\partial c}{\partial ps_{\mathrm{q}}}. \tag{4}$$

However, employing a unity gradient does not entirely capture the quantization characteristics. Furthermore, it is not suitable for gradient-based optimization, as it potentially causes a significant gradient error (red-colored regions in Fig. 3) [23], which can lead to a degradation in the network's accuracy.

To enhance training accuracy, previous works [7], [8] scaled the quantization range of the ADC with a fixed scale factor, q $(0 < q < 1)$, as illustrated in Fig. 3(b). In this approach, the forward and backward operations are defined as:

$$\textbf{Forward: } ps_{\mathrm{q}} = q \cdot sign(ps), \text{ where } q = m \cdot s, \tag{5}$$

$$\textbf{Backward: } \frac{\partial c}{\partial ps} = \frac{\partial c}{\partial ps_{\mathrm{q}}}. \tag{6}$$

Since this technique also adopts the STE method for surrogate gradient in the backward pass, there still exist inherent limitations in precisely approximating the gradients.

Another work in [9], [20] clips the gradient to zero outside the dynamic range with a trainable scale parameter, a, as presented in Fig. 3(c). The operations in this work are defined as:

$$\textbf{Forward: } ps_{\mathrm{q}} = a \cdot sign(ps), \tag{7}$$

$$\textbf{Backward: } \frac{\partial c}{\partial ps} = \begin{cases} \dfrac{\partial c}{\partial ps_{\mathrm{q}}}, & |ps| \le a, \\ 0, & |ps| > a. \end{cases} \tag{8}$$

Effectively, this approach utilizes the clip approximation (one of the STE methods) with the dynamic clip range, a. The derivative of the sign function is set to zero outside $[-a, +a]$ and to unity within the clip range. However, this technique still approximates the impulse with unity, leading to a significant gradient error during backward propagation.

B. Proposed Impulse Gaussian Approximation (IGA)

To address this challenge, we suggest a novel ADC-less training algorithm, impulse Gaussian approximation (IGA). As depicted in Fig. 3(d), the IGA introduces an innovative approach by redefining the impulse gradient to the Gaussian gradient. In the forward propagation, the partial sum is binarized with (1), without an extra scale factor or parameter. In the backward propagation, we implement an alternative gradient utilizing the Gaussian function:

$$G_\sigma(ps) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x}{\sigma}\right)^2}, \tag{9}$$

$$\textbf{Backward: } \frac{\partial c}{\partial ps} = \frac{\partial c}{\partial ps_{\mathrm{q}}}(2m \cdot G_\sigma(ps)). \tag{10}$$

Here, $G_\sigma(\cdot)$ denotes Gaussian function. The surrogate gradient robustly maintains the gradient information of the network, minimizing gradient error by amplifying the gradient behavior around zero, thereby improving the accuracy of training. Unlike incorporating the trainable scale parameter into the computational graph as in [9] and [20], it is noteworthy that the proposed IGA demonstrates remarkable performance despite leveraging a deterministic scale factor, m. The detailed procedure of the IGA is described in the Algorithm 1. In this pseudocode, **NonlinearTransform** consists of **BatchNorm**, **Pooling**, and **Activation** layers. And each operator name with the suffix **BWD** denotes the backward pass of the corresponding forward operator.

Algorithm 1 Training an I-Layer Net With the IGA Technique.

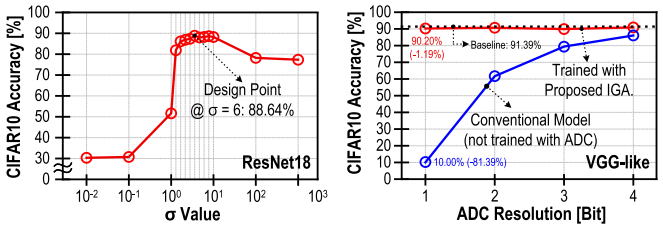
Require: IGA parameter σ; fan-in size of A-IMC array m; Hadamard product $\odot$; input activation tensor IA; weight tensor W; partial sum tensor PS; objective function c; kernel shape of i-layer $K^i \times K^i \times D^i$

return IA^{i+1}
Backward:
for $i = I - 1$ to 2 **do**
  $\frac{\partial c}{\partial PS^i_{\mathrm{q}}} \leftarrow$ **NonlinearTransformBWD** $\left(\frac{\partial c}{\partial IA^{i+1}}, PS^i_{\mathrm{q}}\right)$
  for $j = 1$ to $\left\lceil (K^i \times K^i \times D^i)/m \right\rceil$ **do**
    $\frac{\partial c}{\partial PS^i[j]} \leftarrow \frac{\partial c}{\partial PS^i_{\mathrm{q}}} \odot (2m \cdot G_\sigma(PS^i[j]))$
    $\frac{\partial c}{\partial \tilde{IA}^i_{\mathrm{q}}[j]} \leftarrow$ **ConvBWD** $\left(\frac{\partial c}{\partial PS^i[j]}, \tilde{W}^i_{\mathrm{q}}[j]\right)$
    $\frac{\partial c}{\partial \tilde{W}^i_{\mathrm{q}}[j]} \leftarrow$ **ConvBWD** $\left(\frac{\partial c}{\partial PS^i[j]}, \tilde{IA}^i_{\mathrm{q}}[j]\right)$
  end for
  $\frac{\partial c}{\partial IA^i_{\mathrm{q}}} \leftarrow$ **stack_for_channel** $\left(\frac{\partial c}{\partial \tilde{IA}^i_{\mathrm{q}}}\right)$
  $\frac{\partial c}{\partial W^i_{\mathrm{q}}} \leftarrow$ **stack_for_channel** $\left(\frac{\partial c}{\partial \tilde{W}^i_{\mathrm{q}}}\right)$
  $\frac{\partial c}{\partial IA^i} \leftarrow \frac{\partial c}{\partial IA^i_{\mathrm{q}}} \odot (2 \cdot G_\sigma(IA^i))$
  $\frac{\partial c}{\partial W^i} \leftarrow \frac{\partial c}{\partial W^i_{\mathrm{q}}} \odot (2 \cdot G_\sigma(W^i))$
  $W^i \leftarrow$ **Adam** $\left(W^i, \frac{\partial c}{\partial W^i}\right)$
end for

We evaluate the proposed IGA on BNNs, where all convolutional layers are binarized except for the first and last layers [16], [23]. Note that the IGA is only applied to PS, while the naive STE [22] is applied to input activation (IA) and weight [15]. The hyperparameter of the IGA is the standard deviation σ of the Gaussian function $G_\sigma(\cdot)$. Fig. 4(a) indicates the CIFAR10 accuracy versus σ with the IGA algorithm for ResNet18. When m is set to 128, we found an optimal value of σ to be 6 through empirical studies, achieving a maximum precision of 88.64%. It is noted that the IGA can be applied to a multi-bit ADC scheme by increasing the resolution of the quantization function in the forward pass and overlaying the multiple Gaussian functions at the quantization boundaries in the backward pass. Fig. 4(b) presents a comparison of the CIFAR10 accuracy across different ADC resolutions for VGG-like CNN (128C3-128C3-MP2-256C3-256C3-MP2-512C3-512C3-MP2-1024FC-1024FC-10FC) trained with IGA and with a conventional method (trained without ADC quantization).

The IGA provides consistently robust accuracies across all cases, and experiences only a slight accuracy drop of −1.19% even in the 1-bit scenario. In the absence of ADC-less training,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 4. (a) Training accuracy versus sigma in ResNet18 trained with IGA. (b) Comparison of inference accuracy between the VGG-like CNNs trained with IGA and with the conventional method across different ADC resolutions.

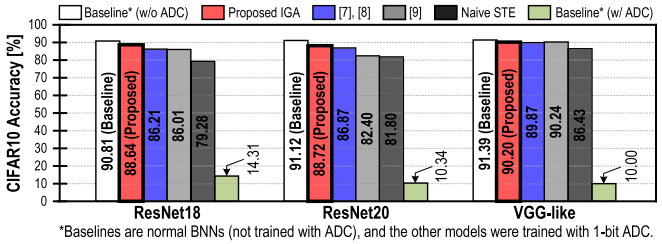
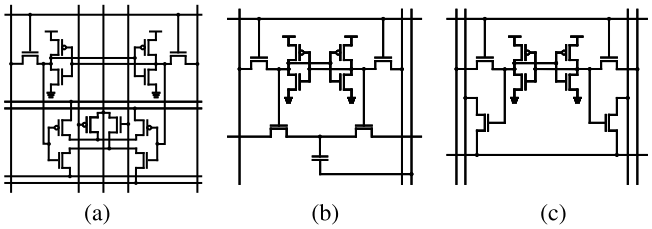


Fig. 5. Accuracy comparison with various ADC-less binarized models (1-bit IA, 1-bit weight, and 1-bit ADC). The ResNet model was trained with 500 epochs, and the VGG-like model was trained with 100 epochs.



(a) (b) (c)

Fig. 6. Various A-IMC SRAM bitcells supporting binary or ternary MAC operation: (a) 12T SRAM [3], (b) 8T1C SRAM [5], and (c) 8T SRAM [6].

accuracy is heavily affected by the ADC resolution, reaching 10.00% at the 1-bit resolution, as ADCs cause severe quantization errors for PS values. From a hardware perspective, applying IGA enables the ADC resolution to be reduced to 1-bit, which is equivalent to a SA of the conventional memory structure. Compared with the flash ADC used in prior A-IMC works [3], [5], this corresponds to about an 1100% improvement (i.e., roughly 11× reduction) in ADC power and area overhead. In particular, for applications that require higher-precision ADCs, the large number of comparators in an A-IMC increases the BL capacitance, which in turn limits the computation speed, computation energy, and the dynamic range of the transfer curve. In contrast, in the proposed IGA scheme, these issues are minimized at the cost of only negligible inference accuracy degradation, thereby improving the overall power, performance, and area of the hardware.

In Fig. 5, we compare the training performance of the proposed IGA with the prior works [7], [8], [9], [20], the naive STE approach, and the conventional method for ResNet18, ResNet20, and VGG-like CNNs. To ensure fair comparisons, the models were trained under identical conditions (learning rate, batch size, number of epochs, activation function, etc.),

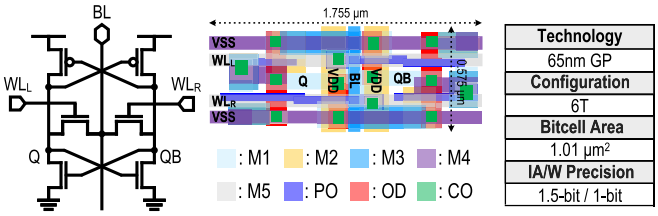


Fig. 7. The proposed binary/ternary compatible 6T SRAM bitcell.

except for the ADC approximation method. Note that the CNNs in this section are binarized models with binary IA and binary weights, and ResNet20 in this paper is a wide version of the original ResNet20, with the number of filters in each convolutional layer increased by a factor of 4. The proposed IGA exhibits the best accuracies of 88.64% and 88.72% in ResNet18, ResNet20, respectively. In the VGG-like CNN, the work in [9], [20] reports a precision of 90.24%, which is 0.04% higher than that of IGA (90.20%). However, it suffers significant accuracy loss as the network depth increases, showing accuracies of 86.01% and 82.40% in ResNet18 and ResNet20, respectively, which are lower by −2.63% and −6.32% compared to the IGA results, demonstrating that the gradient loss from the unity gradient-based approximations impedes the training. Therefore, the proposed IGA is suitable for training deep and realistic network models.

III. 2 KB ADC-Less 6T SRAM-Based IMC

A. Binary/Ternary Compatible 6T SRAM

The schematic and layout of the proposed binary/ternary compatible 6T SRAM are detailed in Fig. 7. It achieves an active area of 1.01 μm^2, representing a 3.88×, 2.88×, and 3.21× reduction compared to prior works [3], [5], [6] that support binary or ternary operations. As in Section I, the proposed bitcell separates wordlines into WL_L and WL_R [10] for ternary multiplication, and unifies BLs to accumulate analog MAC voltages without additional circuits and latency for accumulation operation. Fig. 8 illustrates the operating principle of the bitcell. In Fig. 8(b), with IA = 0, both wordlines (WL_L, WL_R) are active. In this case, one PMOS and one NMOS become active, leading to bypassing currents on a BL that nullify each other, yielding an effective multiplication of 0 (PU + PD ≈ 0). On the other hand, for a multiplication of + 1, only a single side of the WLs with a high storage node is active. For example, as shown in the top of Fig. 8(a), when Q is high and $\bar{Q}$ is low (i.e., weight = +1), WL_L is active and WL_R is connected to ground (i.e., IA = +1), resulting in PU current on a BL. For a multiplication of −1, as shown in Fig. 8(c), only a single side of the WLs with a low storage node is active, causing PD current on a BL, in the opposite manner of the multiplication of + 1. Unlike previous works [3], [4], [5], [6], [11], [13] that support binary or ternary MAC operations, it is noteworthy that the proposed 6T bitcell enables both binary and ternary activations with minimal area, without any additional logic, such as transistors or capacitors. Thanks to its compact area, the proposed A-IMC array achieves macro density of 0.564 Mb/mm^2, exhibiting the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

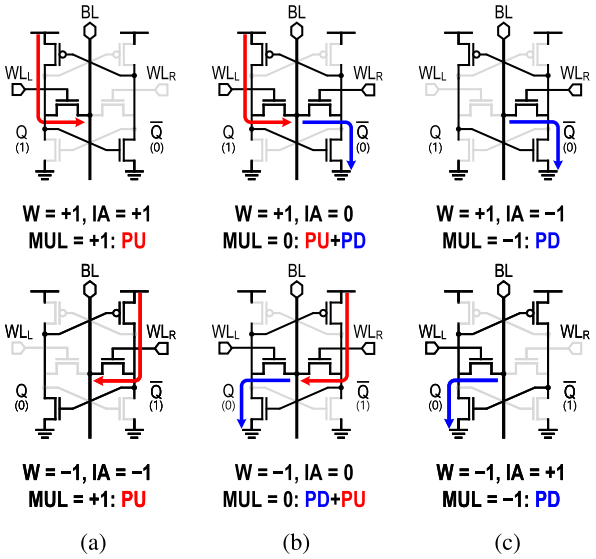


Fig. 8. Binary/ternary multiplication principle with pull-up/pull-down current path. (a) Multiplication of + 1, (b) 0, and (c) −1.

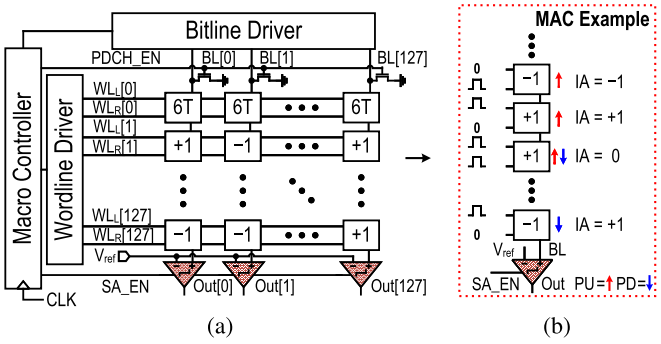


Fig. 9. (a) Overall architecture of the proposed 6T SRAM macro with sense amplifiers and macro controller. (b) MAC example with ternary IA.

highest area efficiency among state-of-the-art 65 nm works in Table II.

B. Overall Architecture

When designing an A-IMC, the macro size is an important design consideration [24]. In an A-IMC that performs fully parallel matrix–vector multiplications (MVMs), the throughput generally scales with the macro size. Furthermore, in ADC-less DNNs, the inference accuracy of the DNN is inversely related to the row size of the A-IMC macro [9]. Under this trade-off, we empirically observed that using a row size of 128 (88.64%) significantly improves the accuracy degradation compared to a row size of 256 (87.02%) for ResNet18, and therefore selected 128 rows in our design. In addition, to maintain a reasonable level of throughput and to enable a fair comparison with prior works whose macro capacity is mostly 2 kB [3], [5], [6], [10], [11], we set the column size to 128.

Fig. 9(a) illustrates the overall 128 × 128 A-IMC macro architecture that combines 6T SRAM bitcells with sense amplifiers, similar to the standard SRAM array. The NMOS transistors on each BL are responsible for pre-discharging,

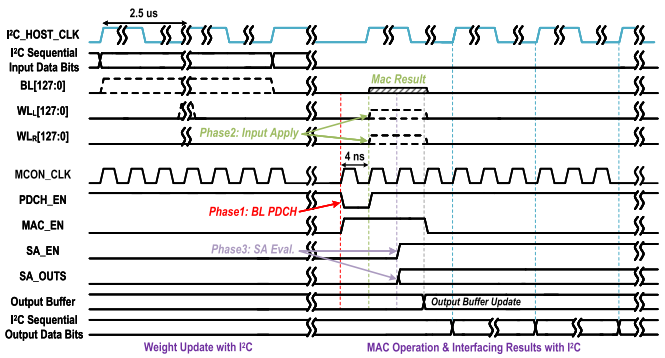


Fig. 10. Overall operation flow of the proposed A-IMC chip. IA/weights are loaded to the A-IMC engine through I²C protocol, and the macro controller coordinates MAC operation followed by output buffering. I²C_HOST_CLK indicates the 400 kHz bitrate of the Aardvark I²C host adapter.

while the macro controller coordinates the MAC procedure, including BL pre-discharge, IA assertion to WLs, and ADC evaluation in a sequential manner. Fig. 9(b) presents a ternary MAC example with a single column where the BL voltage builds up depending on the direction of the current through the PU/PD paths constructed in each bitcell.

To quantitatively analyze the MAC operation of the proposed macro, three types of current are defined: i_{PU}, i_{PD}, i_{PU+PD}, which are the currents of multiplication of + 1, −1, and 0, respectively. When k_1, k_2 are the number of PU cells, PD cells in the single column, the total current from the PU cells, I_{PU}, and the total current from the PD cells, I_{PD}, are defined as follows:

$$I_{PU} = k_1 \cdot i_{PU}, \; I_{PD} = k_2 \cdot i_{PD}. \tag{11}$$

As the number of PU+PD cells corresponds to the bitcells except for the PU and PD cells in the column, the total current of multiplication of 0, I_{PU+PD} can be defined as:

$$I_{PU+PD} = (128 - (k_1 + k_2)) \cdot i_{PU+PD}. \tag{12}$$

Finally, the BL voltage V_{BL}, which is the result of the analog MAC operation, is given by:

$$V_{BL} = V_{SRAM} - R_{BL} \cdot (I_{PU} + I_{PD} + I_{PU+PD}), \tag{13}$$

with the parasitic resistance of the BL wire, R_{BL}.

The operation sequence of the proposed A-IMC chip is presented in Fig. 10. The weights are stored in the array through serial data bits transmitted via the I²C interface and mapped onto the BL and WL_L[1] is asserted, while the right wordline (WL_R) is tied to ground. After weight loading, MAC operation is conducted by the macro controller within three clock cycles (i.e., 12 ns). First, the BLs are pre-discharged by NMOS footers with PDCH_EN, and BL drivers are off by MAC_EN for floating BL. An IA vector is applied by enabling both wordlines of WL_L and WL_R, and based on the bitwise multiplications results, the SRAM bitcells make PU/PD paths on the BLs, thus generating 128 parallel BL voltages that represent the PS results. The PU/PD paths remain until the SA evaluation phase for maintenance of the MAC

[1]During write operation of the SRAM, only the left wordline (WL_L).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

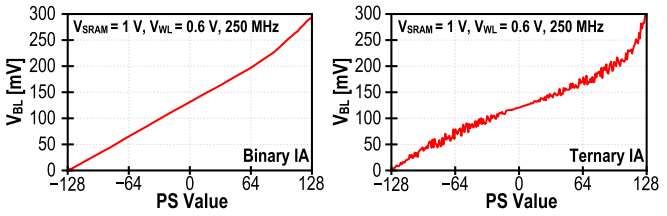


Fig. 11. Simulated voltage transfer curve regarding PS values for (a) binary IA and (b) ternary IA at 250 MHz operating frequency.

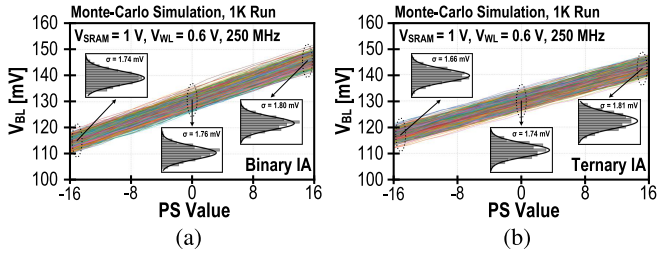


Fig. 12. Simulated voltage transfer curves around PS = 0 with statistical mismatch for (a) binary IA and (b) ternary IA.

voltages. The analog PS outputs on the BLs are digitized by the sense amplifiers (SA_EN). Once the MAC operation finishes, the digital PS outputs are loaded to the output buffer and serially read out as serial data bits through the I^{2}C interface.

Fig. 11 illustrates the simulated voltage transfer curves for binary and ternary MAC operations at an operating frequency of 250 MHz. The transfer curves achieve a dynamic range of $\approx$ 300 mV under a 1 V SRAM supply (V_{SRAM}) and a 0.6 V WL driver supply (V_{WL}). Note that ternary operations exhibit noisy BL voltages when the PS value deviates from zero, even under the same PS value, due to variations in PU/PD ratios caused by different IA/weight data patterns of the DNN.

In the ADC-less A-IMC architecture, the computing precision depends on the linearity and variation of the analog MAC voltage around PS = 0. The Monte Carlo simulation results with the region from PS = −16 to PS = +16 of the proposed 6T SRAM macro are presented in Fig. 12. For both binary and ternary IAs, the proposed macro achieves a standard deviation of V_{BL} less than 2 mV.

IV. Measurement

In this section, we present hardware measurement results and classification accuracies for several DNN models (MLP, ResNet18, and ResNet20) trained with the IGA proposed in Section II. All partial sums in the networks except for the first and last layers are quantized to 1-bit, which corresponds to the SAs in the proposed ADC-less A-IMC. We also show that, compared with a baseline, ADC-less DNNs are more vulnerable to hardware noise. To address this issue, we introduce a PS-wise hardware error-aware training scheme and validate its effectiveness through measurement with the prototypes.

A. Implementation and Characterization of Fabricated Dies

Fig. 13 shows the fabricated chip micrograph of the proposed A-IMC macro in a 65 nm CMOS GP process. The

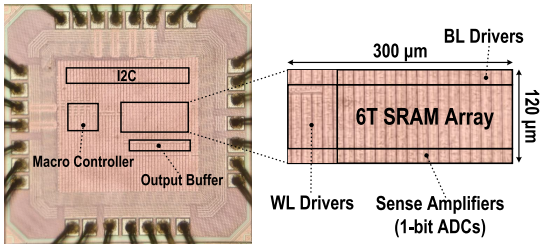


Fig. 13. Chip micrograph with detailed descriptions.

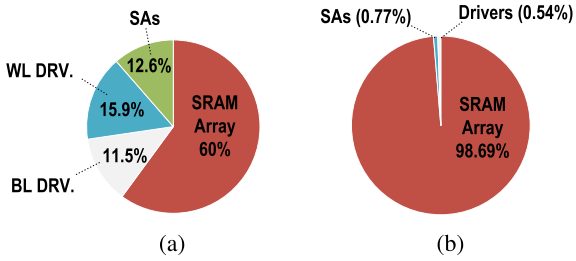


Fig. 14. (a) Area and (b) power breakdown of the proposed A-IMC macro.

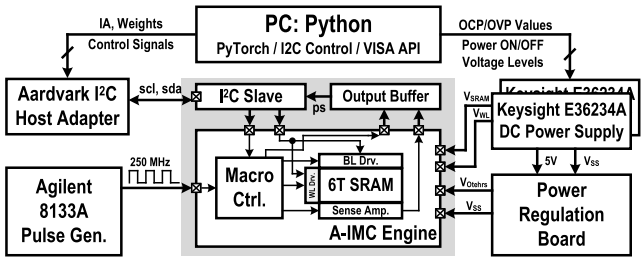


Fig. 15. Measurement setup with detailed equipment description.

300 μm × 120 μm macro integrates a 6T SRAM array, WL/BL drivers, and sense amplifiers, along with peripheral blocks such as the controller, I^{2}C interface, and output buffer. The I^{2}C block enables reliable communication with external controllers for input, weight, and control delivery during evaluation. After the three-cycle operation sequence of precharge, MAC, and SA evaluation illustrated in Fig. 10, the output buffer acts as a cache, holding the SA outputs (i.e., partial sums) before serialization and off-chip communication.

As illustrated in Fig. 14(a), the SRAM array dominates the active area with 60%, while WL drivers, BL drivers, and SAs occupy 15.9%, 11.5%, and 12.6%, respectively. In Fig. 14(b), 98.69% of the power is dissipated by the IMC array, whereas only 0.54% and 0.77% are consumed by the drivers and SAs, respectively. These results indicate that the overhead of the SA-based 1-bit digitizers is almost negligible. As presented in Table I, the proposed IMC exhibits the most competitive performance, achieving an ADC density of 4.708 Mb/mm^2 while demonstrating the lowest ADC power occupancy of 0.77% among recently published SRAM-based IMCs.

Fig. 15 illustrates the measurement setup. The PC running Python manages the overall control, sending inputs and weights through the I^{2}C host adapter. The SRAM array performs MAC operations, and the results are read back via the I^{2}C interface, while the remaining computations are handled in

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

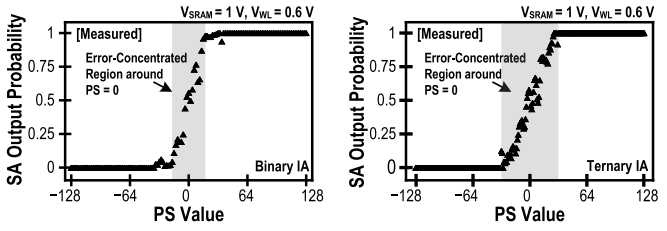


Fig. 16. Measured SA output probability function for (a) binary IA and (b) ternary IA.

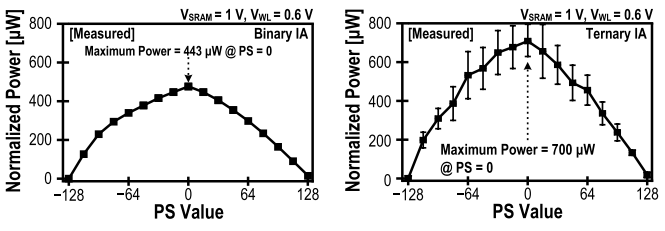


Fig. 17. Measured power consumption versus PS values for (a) binary IA and (b) ternary IA. Note that it is normalized with a fan-out size of 128.

the PC. An external pulse generator (Agilent 8133A) supplies the clock, and a programmable DC power supply (Keysight E36234A) provides regulated voltages through a custom board. This setup enables synchronized functional validation and accurate power measurement.

B. SA Output Probability and Power Dissipation

As shown in Fig. 16, the measured SA output probability results are presented for (a) binary and (b) ternary IAs under V_{SRAM} = 1 V and V_{WL} = 0.6 V. In both cases, errors are concentrated around PS = 0, where the differential input of the SA becomes minimum, thereby reducing the decision margin. The error-concentrated region is also observed to be wider in the ternary case, which is consistent with the simulation results shown in Fig. 11. This degradation arises because ternary operations require simultaneous activation of multiple word-lines, which increases sensitivity to device mismatch and leads to current imbalance. Unlike binary operations with a fixed PU/PD ratio, ternary operations alter the ratio depending on the number of zeros in the input vector, and the resulting variations manifest as noisy BL voltages, particularly when the PS value deviates from zero [see Fig. 11(b)].

The normalized power consumption with a fan-out size of 128 for binary and ternary IAs is revealed in Figs. 17(a) and (b), respectively. At PS = 0, the number of current paths is maximized, resulting in peak power dissipation of 443 μW and 700 μW for the binary and ternary cases, respectively. Similarly, in ternary operations, the number of zeros in the input vector changes the PU/PD ratio, leading to non-deterministic current paths and considerable power variation during operation.

C. Performance Analysis for Various Operating Voltages

The performance and operating condition of the proposed A-IMC macro depend on the supply voltages of the SRAM and the WL driver. We measured the energy efficiencies and operating regions of the prototype with different supply voltages: 0.80−1.20 V for SRAM and 0.45−0.75 V for the WL driver, for binary IA and ternary IA at an operating frequency of 250 MHz. In Fig. 18, the operating region of the macro is presented. Excessive WL voltages induce BL disturbance and demand high SRAM supply voltages to maintain reliable V_{BL}. If the V_{SRAM} is insufficient, the sensing yield of the macro drastically decreases, corresponding to the fail region

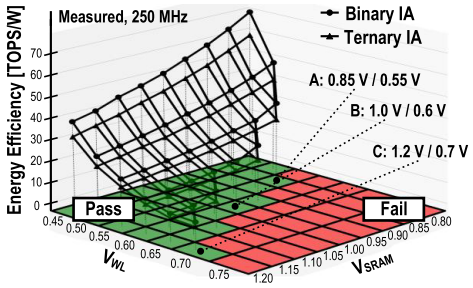


Fig. 18. Measured energy efficiencies and operating conditions for different supply voltage levels of SRAM and WL driver at 250 MHz clock frequency.

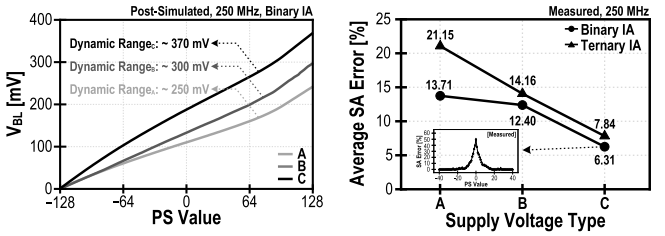


Fig. 19. (a) Simulated voltage transfer curves and (b) measured average SA error probabilities for different voltage setups. Note that the errors were measured using 1000 random IA/W data points following a uniform distribution and reflect the average error in the PS region [−40, + 40], which occupies most of the PS in the DNNs used for the measurement.

in Fig. 18. With higher supply voltages, computation accuracy improves at the expense of energy efficiency. It is noted that the ternary MAC operation consumes more energy because there are more current paths than the binary operation, as mentioned in Section III-A and Section IV-B [see Figs. 8 and 17]. To analyze the trade-off between supply voltage and computation accuracy, we set the representative supply voltage setups: A (0.85/0.55 V), B (1.0/0.6 V), and C (1.2/0.7 V). Fig. 19(a) indicates the simulated voltage transfer curves with the voltage setups.

High supply voltages make the dynamic range of the transfer curves wider: ≈ 250 mV at A, ≈ 300 mV at B, and ≈ 370 mV at C. The wide dynamic range leads to a large sensing margin with a higher decision threshold of the sense amplifier, ≈ 110 mV at A, ≈ 130 mV at B, and ≈ 190 mV at C, respectively. The measured average SA error probabilities for the three voltage setups are described in Fig. 19(b). The maximum voltage case C exhibits a minimum SA error of 6.31%, and the minimum voltage case A exhibits a maximum

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

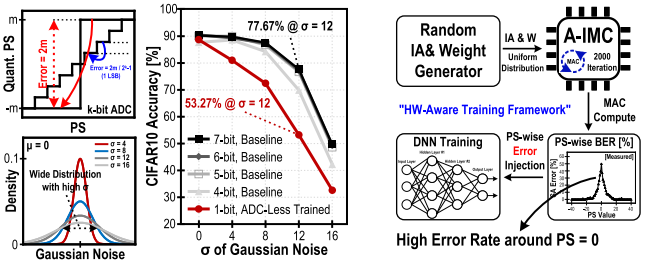


Fig. 20. (a) Noise-vulnerable nature of ADC-less DNNs and (b) illustration of PS-wise hardware error-aware training framework.

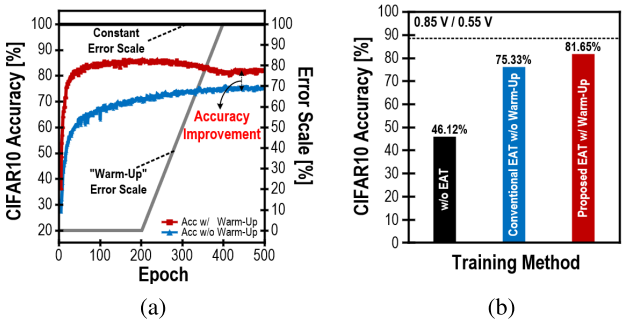


Fig. 21. (a) Proposed error-aware training algorithm with warm-up process and (b) classification accuracies of the various error-aware training schemes.

SA error of 13.71%. The computation error of the ternary operation is larger than that of the binary operation and increases linearly with the voltage settings, whereas the binary operation exhibits robust characteristics. This is because the ternary IA results in more disturbed PS outputs than binary IA, as discussed in Section III-B and Section IV-B.

D. Error-Aware Training (EAT) on Fabricated Chips

In general, the DNN accuracy evaluated on A-IMC suffers from computing errors caused by noise and PVT variations [3], [4], [5], [6], [10], [11], [12]. As shown in Fig. 20, ADC-less DNN is particularly vulnerable to computing errors. The comparison of the inference accuracy with various noise strengths between baseline and ADC-less DNN is presented in Fig. 20(a). The ADC-less DNN exhibits a higher drop than baselines with high-resolution ADCs. To compensate for the accuracy drop from the hardware faults and the resulting computation errors, we trained the DNNs with PS-wise errors obtained from the prototype, as illustrated in Fig. 20(b). However, during error-aware training (EAT), the training is interrupted if the errors are injected from the beginning of the training. We propose a novel EAT method based on warm-up error injection. In Fig. 21(a), out of a total of 500 epochs, the first 200 epochs are trained without errors for stable weight updates, and in the following 200 epochs, the error scale is slowly increased to adapt the model to the errors. Finally, the remaining 100 epochs are trained with a full error scale to enhance accuracy. As shown in Fig. 21(b), with the proposed EAT, the accuracy is significantly improved compared to without EAT and conventional EAT scenarios.

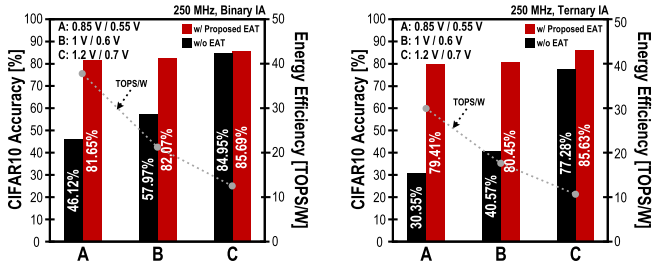


Fig. 22. Measured inference accuracies and energy efficiencies for different supply voltage setups with and without proposed error-aware training at 250 MHz clock frequency: (a) binary IA and (b) ternary IA.

TABLE I
DNN CLASSIFICATION RESULTS OF THE PROPOSED ADC-LESS AND EAT APPLIED A-IMC FOR THE MNIST AND CIFAR10 DATASETS

	DNN Model	Input Activation	SW Baseline[a]	HW Measured[b]	Accuracy Drop
MNIST	MLP	Binary	97.94	97.84	−0.10
		Ternary	98.20	97.72	−0.48
CIFAR10	ResNet18	Binary	88.64	85.69	−2.95
		Ternary	89.05	85.63	−3.42
	ResNet20	Binary	88.72	86.89	−1.83
		Ternary	89.71	85.52	−4.19

[a] Trained with the proposed IGA (1-bit partial sum quantization).
[b] ADC-less with error-aware training (EAT) applied.

The measured classification accuracies with the proposed IGA and EAT at three supply voltage setups are presented in Fig. 22. The proposed A-IMC macro achieves CIFAR10 accuracies of 85.69% and 85.63% for binary and ternary IA at 1.2/0.7 V supply with ResNet18. Although there is a trade-off between energy efficiency and accuracy, the worst-case results are 81.65% and 79.41% in binary and ternary IA at 0.85/0.55 V supply, demonstrating better performance than previous ADC-less A-IMC work [10] without in-memory calibration and batch normalization, reducing the throughput of the A-IMC macro.

Table I summarizes inference accuracies measured on the proposed A-IMC prototype. For MNIST/MLP (784-512-512-512-10), the accuracy drops are 0.10% (binary) and 0.48% (ternary). For the CIFAR10 dataset, classification accuracies on ResNet18 present declines of 2.95% (binary) and 3.42% (ternary). On ResNet20, the accuracies exhibit declines of 1.83% (binary) and 4.19% (ternary), respectively. As discussed in conjunction with Fig. 11(b), the nonlinearity in the ternary IA case introduces additional computation errors in the A-IMC macro [see Fig. 16(b) and Fig. 19(b)]. Consequently, even though the ternary networks achieve higher baseline accuracy than the binary counterparts in software, their measured hardware accuracy becomes slightly lower due to this effect.

Fig. 23 shows the boxplots of the 10 test chips at V_{SRAM} = 1 V, V_{WL} = 0.6 V, at 250 MHz clock frequency. Ternary IA indicates a higher median and wider spread of SA error, with outliers near PS = 0. We selected the 3rd and the 8th

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

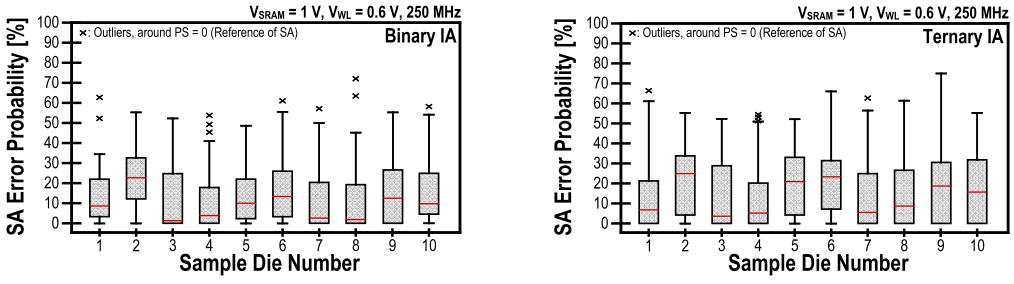


Fig. 23. Box plots of the 10 test chips: (a) binary IA and (b) ternary IA.

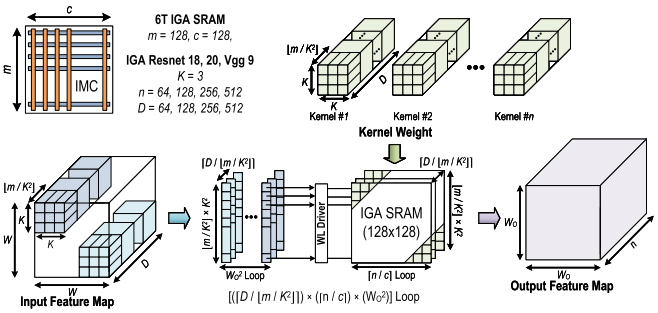


Fig. 24. Overall mapping scheme that transforms 3×3 convolutions into matrix–vector operations on the 128×128 6T IGA-SRAM IMC array along channel-wise grouping and 3×3 kernel unrolling.

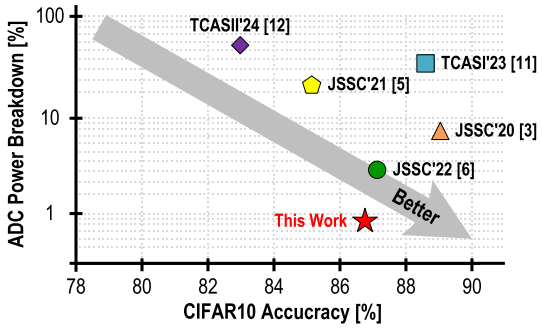


Fig. 25. Comparison of ADC power versus CIFAR10 accuracy with previous A-IMC works [3], [5], [6], [11], [12]. The proposed work (indicated by a red star) achieves the lowest ADC power share with competitive accuracy.

chips for the best cases to measure the image classifications in Table I and used them to characterize hardware-induced computation errors caused by circuit faults for the proposed EAT algorithm.

E. DNN Mapping Methodology

As illustrated in Fig. 24, the overall mapping scheme transforms 3×3 convolutions into MVM on the proposed 128×128 6T IGA-SRAM IMC array through channel-wise grouping and 3×3 kernel unrolling, in line with prior works that cast convolutional layers into an MVM forms for accelerator mapping [25]. The array has $m = 128$ WL and $c = 128$ BL. For a 3×3 kernel ($K = 3$), the input feature map is first partitioned into channel groups of size

$$C_{\text{sub}} = \left\lfloor \frac{m}{K^2} \right\rfloor, \tag{14}$$

and the corresponding unrolled kernel vector length is bounded by

$$C_{\text{sub}} K^2 \le m, \tag{15}$$

which ensures that it fits within the WL dimension. When $C_{\text{sub}} K^2 < m$, the remaining $m - C_{\text{sub}} K^2$ WL positions are populated with dummy cell pairs whose weight and input products approximately sum to zero, so that their net contribution to the BL current largely cancels out and only introduces a negligible perturbation compared to the valid MAC term. The resulting $C_{\text{sub}} \times 3 \times 3$ patches are flattened to length $C_{\text{sub}} K^2$, mapped onto the active WL, and applied as input vectors along the WL, while a kernel chunk of size n_k ($n_k \le c$) is mapped onto the BL such that each active BL stores one unrolled kernel vector (the remaining BL, if any, is left idle). For a layer with D input channels and n output channels, the computation is completed by iterating over $\lceil D/C_{\text{sub}} \rceil$ input-channel groups, $\lceil n/c \rceil$ kernel chunks, and all output-pixel locations in the feature map, and accumulating the partial sums from different input groups to reconstruct the full dot products. This mapping is uniformly applied to all CNN layers, enabling reuse of the same 128×128 IMC macro while maintaining equivalence with the original 3×3 convolution. Unlike classical systolic arrays [26], which map computations onto a 2-D grid of processing elements with spatio-temporal data streaming, the proposed IMC macro performs MAC operations directly inside a static memory array.

F. Comparison

Table II provides a comparative overview of this work against recent A-IMC macros. In prior SRAM implementations, improvements in inference accuracy came at the expense of significant ADC overhead [3], [5], [11], [13]. Some works [6], [12] tried to reduce the area of the ADC with 1–5 bit column ADCs, but this approach sacrifices computing throughput and demands significant energy consumption for a precise digital conversion of the analog MAC voltage. By contrast, this work adopts an innovative design that maintains accuracy even with 1-bit SA by co-optimizing the hardware architecture and DNN training flow to utilize 1-bit partial sum quantization. As a result, the proposed ADC-less A-IMC engine achieves a notable ADC density of 4.708 Mb/mm^2 with

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE SUMMARY AND COMPARISON OF RECENT PUBLISHED SRAM-BASED A-IMCS

	This work	VLSI'19 Kim [10]	JSSC'20 Yin [3]	JSSC'20 Jiang [5]	JSSC'21 Chen [13]	JSSC'22 Yu [6]	TCASI'23 Cheon [11]	TCASII'24 Shah [12]
Technology [nm]	**65 (GP)**	28 (LPP)	65 (LP)	65 (LP)	65 (LP)	65 (LP)	28 (LPP)	65 (LP)[f]
SRAM Configuration	**6T**	6T	12T	8T1C	6T	8T	10T	6T
Operating Voltage [V]	**0.8−1.2 0.5−0.7**	1.0	0.6−1.0	1.0/0.8/0.6	1.2	0.8/0.45	0.6−1.0	1.2/0.67
Macro Capacity [KB]	**2**	2	2	2	8	2	2	1.5
Macro Area [mm^2]	**0.0290**	0.0202[a]	0.1125	0.081	0.1749	0.07[a]	0.0385	0.0248
Operation Domain	**Voltage**	Voltage	Voltage	Charge	Charge	Current	Charge	Charge
Input Precision	**1/1.5**	1	1.5	1	1−8	1	1.5	1
Weight Precision	**1**	1	1	1	1/2/3/4/5/8	1	1	1
ADC Precision	**1**	1	3.46	5	7	1−5	3.32	1−5
ADC Density [Mb/mm^2]	**4.708**	10.67	0.0940	1.324	-	2.926	3.084	1.238
ADC Power Breakdown [%]	**0.77**	-	7.52	22	-	3.4	46.7	12[d]−62[e]
Macro Density [Mb/mm^2]	**0.564**	0.810	0.146	0.202	0.375	0.234	0.426	0.495
Clock Frequency [MHz]	**25−250**	25	200−1200	50	70	200	333	370
Throughput [GOPS]	**1365**	615.6	614.4	1638	573.4	7840[d]−252[e]	2294	6002[d]−188[e]
Energy Efficiency [TOPS/W]	**37.8**	300.3	403	671.5	49.4	490[d]−15.8[e]	2941[b]	984[bd]−79[e]
Area Efficiency [TOPS/mm^2]	**47.06**	30.48	5.461	20.22	3.4	112[d]−3.6[e]	59.584	242[d]−7.6[e]
Error-Aware Training	○	×	×	×	×	×	×	×
MNIST Accuracy [%]	**97.84/97.72**	98.42	98.84	98.30	98.80	96.37[e]	98.66	96.50[ce]
CIFAR10 Accuracy [%]	**86.89[c]/85.63[c]**	79.38	88.78[c]	85.50	89.00	87.14[e]	88.48	83.10[ce]

[a] Estimated from chip photo or layout figure. [b] Post-layout simulation. [c] Measured from output probability.
[d] Measured with sense amplifier (ADC-less). [e] Measured with 5-bit ADC. [f] Simulation only (not fabricated).

comparable inference accuracy of 86.89% for the CIFAR10 dataset, which is higher than that of prior arts [5], [12] using high-resolution ADCs. Note that although the previous ADC-less A-IMC macro [10] reported an inference accuracy of 79.38% for the CIFAR10 dataset, utilizing in-memory calibration and batch normalization at the expense of throughput, this work further improves the accuracy using the EAT algorithm in Fig. 20(b) without sacrificing throughput. Fig. 25 shows the ADC power share versus CIFAR10 accuracy for this work and state-of-the-art A-IMCs. While prior works suffer from high ADC power overheads for only incremental accuracy benefit, this work stands out, achieving a competitive accuracy of 86.89% while consuming just 0.77% of the total power in the ADC.

From the bitcell perspective, unlike [3] and [11], the proposed 6T SRAM implements ternary activation using only six transistors, without incorporating any additional transistors or passive components within the bitcell. Moreover, in contrast to [3], [5], [10], [11], [13], MAC operation is realized solely through unified BL, without adding extra devices either at the bitcell or array. Thanks to the compact binary/ternary compatible 6T SRAM and ADC-less architecture, the proposed macro attains a decent macro density of 0.564 Mb/mm^2 and area efficiency of 47.06 Mb/mm^2 with throughput of 1365 GOPS, exhibiting remarkable performance among the previous 65 nm binary/ternary A-IMC works [3], [5], [6], [12], [13].

Despite its high area efficiency due to ADC-less architecture, the proposed A-IMC macro reports a relatively large energy cost. This is mainly attributed to the large crowbar current in the SRAM array during voltage-mode MAC operation, which constitutes a major source of power dissipation. In addition, the use of a general-purpose (GP) process further contributes to the overall power consumption because of the high drive current characteristics of the transistors. Implementing the proposed A-IMC engine in a low-power (LP) process is therefore expected to improve energy efficiency.

V. CONCLUSION

This work presents a compact and efficient 6T SRAM-based A-IMC engine designed for binary and ternary neural networks. By integrating a 1-bit partial-sum quantization into both hardware and training algorithm design, we significantly reduce area and power overhead while preserving high inference accuracy. The proposed IGA effectively overcomes the gradient vanishing problem inherent in ADC quantization with a Gaussian function, enabling ADC-less training that minimizes accuracy degradation even under 1-bit quantization. The proposed macro implements ternary MAC operations using a compact 6T SRAM bitcell with no additional logic such as transistors or capacitors. Furthermore, by leveraging sense amplifiers as lightweight quantizers, the architecture maintains a similar array structure to conventional SRAM caches, ensuring compatibility and scalability with existing memory architecture. The fabricated 2 KB A-IMC engine in 65 nm CMOS GP technology achieves 47.06 TOPS/mm^2 compute density, exhibiting the best performance compared to the state-of-the-art 65 nm works in Table II, and 37.8 TOPS/W energy efficiency with a peak throughput of 1365 GOPS at 250 MHz clock frequency. Despite the lack of quantization information due to ADC-less architecture, the prototype demonstrates robust performance on the MNIST and CIFAR10 datasets.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

APPENDIX A
HYPERPARAMETERS AND TRAINING EFFICIENCY OF THE PROPOSED ADC-LESS TRAINING ALGORITHM

All experiments were conducted on a Linux workstation equipped with an Intel Xeon w5-3433 CPU (16 cores, 32 threads) and an NVIDIA GeForce RTX 4090 GPU with 24 GB of VRAM. We used PyTorch (2.5.1) with CUDA (12.4) and cuDNN (9.1.0) on a single GPU. For reproducibility, we fixed the random seed to 42 in Python, NumPy, and PyTorch (including CUDA), and enabled deterministic cuDNN operations when possible.

A. Training Settings and Hyperparameters

Our training pipeline is based on the original BNN implementation [15]; we reuse the same data loading, preprocessing, logging, and checkpointing framework, and modify only the components required by the proposed ADC-less training.

We use standard data augmentation (random cropping and horizontal flipping). Unless otherwise noted, we use the following common hyperparameters:

- Number of training epochs: 500
- Mini-batch size: 128
- Optimizer: Adam
  - initial learning rate 5×10^{-3}
  - $\beta_1 = 0.9$, $\beta_2 = 0.999$
  - weight decay = 0
- Learning-rate schedule: cosine-annealing scheduler with $T_{\max} = 500$ epochs

Apart from above settings, all remaining training-loop details follow the original BNN code.

B. Hyperparameter of the Proposed IGA

As briefly mentioned in Section II, the main hyperparameter of IGA is σ, which determines the shape of the Gaussian. Since the training accuracy is highly sensitive to σ, we empirically searched for an optimal value across various DNN models.

As shown in Fig. 4(a), the optimal σ was found to be 6, and similar trends were observed for the other models. Experiments were conducted assuming a row size of 128, corresponding to the fabricated chip. It is noted that the optimal value of σ can vary depending on other conditions, such as the row size of A-IMC macro, the DNN model, or the input and weight precision.

C. Run-Time and GPU Memory Utilization

To quantify the computational cost of the proposed ADC-less training (IGA) relative to a baseline [15], we measured the average training time per epoch and the average GPU memory utilization during training a binary ResNet18 on CIFAR10 dataset.

The measured averages are:

- Average training time per epoch
  - Baseline: 24.53 s
  - IGA: 36.44 s
- Average GPU memory usage
  - Baseline: 1.48 GB
  - IGA: 3.68 GB

The proposed IGA-based ADC-less training increases the per-epoch training time by approximately $\approx 1.49\times$ and the average GPU memory consumption by approximately $\approx 2.49\times$, compared with the baseline that does not model 1-bit partial sums during training.

The higher runtime and GPU memory usage of the IGA mainly come from the explicit PS quantization. In the baseline, the group-wise partial sums from each convolution are directly accumulated into the output. With IGA, every partial sum is passed through a custom autograd function that stores the full-precision partial sums and, in the backward pass, computes an element-wise Gaussian-shaped surrogate gradient. These additional tensors and element-wise operations naturally increase both the training time and the GPU memory utilization.

APPENDIX B
DNN MAPPING METHODOLOGY

A. Details of Tensor Division and Output Size

- $\left\lfloor \frac{m}{K^2} \right\rfloor$: channel depth of the divided tensor.
- $\left\lfloor \frac{m}{K^2} \right\rfloor \times K^2$: length of the unrolled kernel.
- $\left\lceil \frac{D}{\lfloor m/K^2 \rfloor} \right\rceil$: number of divided tensors per kernel.
- Output size W_O for a square kernel:

$$W_O = \left\lfloor \frac{W + 2 \times \text{Padding} - K}{\text{Stride}} \right\rfloor + 1.$$

B. IMC Operation Count for a Single Layer

- Input loop: $\left\lceil \frac{D}{\lfloor m/K^2 \rfloor} \right\rceil$.
- Weight loop: $\left\lceil \frac{n}{c} \right\rceil$.
- Pixel count: ${W_O}^2$.
- Total IMC operations for a single layer:

$$N_{\text{IMC}} = \left\lceil \frac{D}{\lfloor m/K^2 \rfloor} \right\rceil \times \left\lceil \frac{n}{c} \right\rceil \times {W_O}^2.$$

C. Details of the Mapping for a Single Layer

This appendix provides additional explanation for Fig. 24. In our implementation, the IGA-SRAM IMC array has $m = 128$ wordlines and a 3×3 convolution kernel ($K = 3$). Thus, the channel depth of each divided tensor is

$$\left\lfloor \frac{m}{K^2} \right\rfloor = \left\lfloor \frac{128}{9} \right\rfloor = 14,$$

so the input channels D are processed in groups of 14 channels. The resulting number of divided tensors per kernel is $\lceil D/14 \rceil$, and any remaining channels are padded with dummy data whose values are chosen to have negligible impact on the bitline voltage. For example, when $D = 64$, we have $64 = 14 \times 4 + 8$, For each divided tensor, the unrolled kernel

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.

length becomes $14 \times K^2 = 14 \times 9 = 126$ rows, so the remaining two rows in the 128-row array are filled with dummy data. And the last divided tensor contains only 8 valid channels and uses $8 \times 9 = 72$ rows, while the remaining rows are again filled with dummy data.

Given the input feature-map size W, padding, kernel size K, and stride, the output size for a square kernel is

$$W_{\mathrm{O}} = \left\lfloor \frac{W + 2 \times \mathrm{Padding} - K}{\mathrm{Stride}} \right\rfloor + 1,$$

and the total number of output pixels per feature map is W_{O}^2.

On the weight side, the IGA-SRAM array provides $c = 128$ columns, so the output channels n are mapped in groups of 128 channels. The number of weight loops is therefore $\lceil n/c \rceil$. For instance, when $n = 256$, we perform two weight loops with 128 channels each, whereas $n = 512$ requires four loops.

The IMC operation count for a single convolution layer can be decomposed into three factors: the number of input loops, the number of weight loops, and the number of output pixels, similar in spirit to the loop-based cost models used for BNN accelerators in [25], but specialized here to a 128×128 IMC array. Due to the finite row dimension m of the IMC array, at most $\lfloor m/K^2 \rfloor$ input channels can be processed in a single MVM for a $K \times K$ kernel. Therefore, the input channels D are partitioned into $\lceil D/\lfloor m/K^2 \rfloor \rceil$ groups, which determines the input-loop count. For each such group of input channels, a corresponding group of weights is mapped onto the IMC array, forming an input–weight pair that is reused across all spatial locations of the output feature map, the same weights are applied repeatedly for ${W_{\mathrm{O}}}^2$ output pixels. Similarly, along the column dimension, the array provides c columns so that up to c output channels can be computed in parallel. When the number of output channels n exceeds c, the computation is further split along the column dimension: the first c output channels are evaluated in the first pass, and additional passes are performed over the same input activations for the remaining channels. As a result, the output channels n are handled in $\lceil n/c \rceil$ groups, which gives the weight-loop count. For a square feature map, the total number of spatial positions at which the convolution is evaluated is ${W_{\mathrm{O}}}^2$. Combining these three factors, the total number of IMC MVM operations required for a single layer is

$$N_{\mathrm{IMC}} = \left\lceil \frac{D}{\lfloor m/K^2 \rfloor} \right\rceil \times \left\lceil \frac{n}{c} \right\rceil \times {W_{\mathrm{O}}}^2.$$

ACKNOWLEDGMENT

This chip fabrication and EDA tool were supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

[1] M. Shafique, T. Theocharides, V. J. Reddy, and B. Murmann, "TinyML: Current progress, research challenges, and future roadmap," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 1303–1306.

[2] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.

[3] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.

[4] B. Zhang et al., "PIMCA: A programmable in-memory computing accelerator for energy-efficient DNN inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 5, pp. 1436–1449, May 2023.

[5] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: In-memory-computing SRAM macro based on capacitive-coupling computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.

[6] C. Yu, T. Yoo, K. T. C. Chai, T. T.-H. Kim, and B. Kim, "A 65-nm 8T SRAM compute-in-memory macro with column ADCs for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 57, no. 11, pp. 3466–3476, Nov. 2022.

[7] Y. Kim, H. Kim, D. Ahn, and J. Kim, "Input-splitting of large neural networks for power-efficient accelerator with resistive crossbar memory array," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Design (ISLPED)*, Jul. 2018, pp. 1–6.

[8] Y. Kim, H. Kim, J. Park, H. Oh, and J.-J. Kim, "Mapping binary ResNets on computing-in-memory hardware with low-bit ADCs," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 856–861.

[9] U. Saxena, I. Chakraborty, and K. Roy, "Towards ADC-less compute-in-memory accelerators for energy efficient deep learning," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2022, pp. 624–627.

[10] J. Kim et al., "Area-efficient and variation-tolerant in-memory BNN computing using 6T SRAM array," in *Proc. Symp. VLSI Circuits*, Jun. 2019, pp. C118–C119.

[11] S. Cheon, K. Lee, and J. Park, "A 2941-TOPS/W charge-domain 10T SRAM compute-in-memory for ternary neural network," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 2085–2097, May 2023.

[12] I. Shah, K. Sarfraz, and M. Chan, "An efficient 6TP SRAM-based CIM macro with column ADCs for binarized neural networks," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 5, pp. 2509–2513, May 2024.

[13] Z. Chen et al., "CAP-RAM: A charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN inference," *IEEE J. Solid-State Circuits*, vol. 56, no. 6, pp. 1924–1935, Jun. 2021.

[14] M. Courbariaux, Y. Bengio, and J. David, "BinaryConnect: Training deep neural networks with binary weights during propagations," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2015, pp. 3123–3131.

[15] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to + *or* -," 2016, *arXiv:1602.02830*.

[16] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: Imagenet classification using binary convolutional neural networks," 2016, *arXiv:1603.05279*.

[17] H. Alemdar, V. Leroy, A. Prost-Boucle, and F. Pétrot, "Ternary neural networks for resource-efficient AI applications," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2017, pp. 2547–2554.

[18] D. Wan et al., "TBN: Convolutional neural network with ternary inputs and binary weights," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 322–339.

[19] A. Kneip and D. Bol, "Impact of analog non-idealities on the design space of 6T-SRAM current-domain dot-product operators for in-memory computing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1931–1944, May 2021.

[20] S. Negi et al., "Algorithm hardware co-design for ADC-less compute in-memory accelerator," *IEEE Trans. Circuits Syst. Artif. Intell.*, vol. 1, no. 2, pp. 191–203, Dec. 2024.

[21] M. Bertuletti, I. M. Noz-Martín, S. Bianchi, A. G. Bonfanti, and D. Ielmini, "A multilayer neural accelerator with binary activations based on phase-change memory," *IEEE Trans. Electron Devices*, vol. 70, no. 3, pp. 986–992, Mar. 2023.

[22] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.

[23] H. Qin et al., "Forward and backward information retention for accurate binary neural networks," 2019, *arXiv:1909.10788*.

[24] M. Yayla, F. Frustaci, F. Spagnolo, J.-J. Chen, and H. Amrouch, "Unlocking efficiency in BNNs: Global by local thresholding for analog-based HW accelerators," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 4, pp. 940–955, Dec. 2023.

[25] M. Yayla, C. Latotzke, R. Huber, S. Iskif, T. Gemmeke, and J.-J. Chen, "DAEBI: A tool for data flow and architecture explorations of binary neural network accelerators," in *Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS 2023)*. Cham, Switzerland: Springer, 2023, pp. 107–122.

[26] Kung, "Why systolic architectures?," *Computer*, vol. 15, no. 1, pp. 37–46, Jan. 1982.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.



Shin-Uk Kang received the B.S. and M.S. degrees from the School of Electrical Engineering, Hanyang University, Ansan, South Korea. He is currently pursuing the Ph.D. degree with the School of Electrical Engineering, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, South Korea. His research interests include artificial intelligence hardware design and in-memory computing (IMC).



Dong-Hyun Lee (Graduate Student Member, IEEE) received the B.S. degree from the School of Electrical and Electronic Engineering, Dankook University, Jukjeon, South Korea. He is currently pursuing the M.S. degree with the School of Electrical Engineering, Hanyang University, Ansan, South Korea. His research interests include artificial intelligence hardware design and in-memory computing (IMC).



Min-Gwon Song received the B.S. degree from the School of Electrical Engineering, Hanyang University, Ansan, South Korea. He is currently pursuing the M.S. degree with the Department of Electronic Engineering, Hanyang University, Seoul, South Korea. His research interests include artificial intelligence hardware design.



Mingoo Seok (Senior Member, IEEE) received the B.S. degree (summa cum laude) in electrical engineering from Seoul National University, Seoul, South Korea, in 2005, and the M.S. and Ph.D. degrees in electrical engineering from the University of Michigan, Ann Arbor, MI, USA, in 2007 and 2011, respectively.

He was a Member of the Technical Staff with Texas Instruments Inc., Dallas, TX, USA, in 2011. Since 2012, he has been with Columbia University, New York, NY, USA, where he is currently an Associate Professor of electrical engineering. His current research interests include ultra-low-power SoC design for emerging intelligent systems, machine learning VLSI architecture and circuits, variation, voltage, aging, thermal-adaptive circuits and architecture, on-chip integrated power circuits, and nonconventional hardware design, including in-memory computing and analog-mixed-signal computing hardware.

Prof. Seok is/was the Technical Program Committee Member of several conferences, including the IEEE International Solid-State Circuits Conference (ISSCC) and the ACM/IEEE Design Automation Conference (DAC). He has received the 1999 Distinguished Undergraduate Scholarship from Korea Foundation for Advanced Studies, the 2005 Doctoral Fellowship from Korea Foundation for Advanced Studies, and the 2008 Rackham Pre-Doctoral Fellowship from the University of Michigan. He has also received the 2009 AMD/CICC Scholarship Award for picowatt voltage reference work, the 2009 DAC/ISSCC Design Contest for the 35-pW sensor platform design, the 2015 NSF CAREER Award, the 2019 Qualcomm Faculty Award, and the Best Paper Award from IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS in 2022. He serves/served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS from 2013 to 2015, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS since 2015, and IEEE SOLID-STATE CIRCUITS LETTERS from 2017 to 2022. He has also served as a Guest Editor for IEEE JOURNAL OF SOLID-STATE CIRCUITS. He was selected as a Distinguished Lecturer of the Solid-State Circuits Society (SSCS) from 2023 to 2025.



Min-Seong Choo (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical and computer engineering from Seoul National University, Seoul, South Korea, in 2012 and 2019, respectively.

In 2019, he was a Post-Doctoral Researcher with the Inter-University Semiconductor Research Center, Seoul National University. From 2019 to 2020, he was a Research Scholar with the Center for Nanotechnology, NASA Ames Research Center, Moffett Field, CA, USA. From 2020 to 2022, he was a Post-Doctoral Research Scientist with Columbia University, New York, NY, USA. Since 2022, he has been an Assistant Professor with the School of Electrical Engineering, Hanyang University. His research interests include high-speed transceivers (TRX), DSP-based TRX, optical interface, phase-locked loops (PLL), clock and data recovery (CDR), injection-locked oscillator (ILO), memory system architecture, and in-memory computing (IMC).

Dr. Choo has served as a Reviewer for various journals, including IEEE JOURNAL OF SOLID-STATE CIRCUITS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMS, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, and IEEE ACCESS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:13:47 UTC from IEEE Xplore. Restrictions apply.